

WatchDiary

Full Stack Build Blueprint

Movies · Series · Anime — Personal Watch Tracker

Stack: Next.js 14 · PostgreSQL · Prisma · NextAuth.js · Tailwind CSS
Deployment: Vercel + Supabase · Version 1.0 · 2026

What is WatchDiary?	A full stack web app where users log movies, series, and anime they have watched. Each user gets a personal watchlist.
Who is this for?	Almeda — building this as a portfolio project to demonstrate full stack skills end to end: database design, authentication, and deployment.
Tech Stack	Next.js 14 (frontend + backend) · Prisma ORM · PostgreSQL on Supabase · NextAuth.js (OAuth) · Tailwind CSS

1. Current Status

Here is exactly where the project stands right now and what has already been completed.

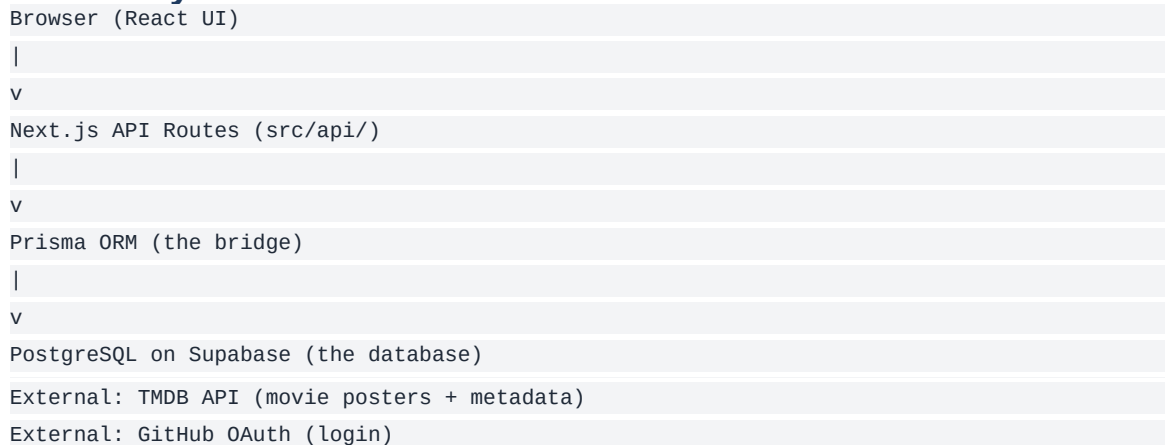
Item	Status	Notes
Next.js 14 project created	DONE	Located at C:\Users\Almeda\Desktop\watchlist
Prisma 7 installed	DONE	Using prisma.config.ts for configuration
Supabase project created	DONE	Project ID: arxmvziepsgrsgdsshke
Database connected	DONE	Tables pushed successfully via npx prisma db push
Database schema written	DONE	User, Entry, Account, Session tables created
GitHub OAuth setup	NEXT	Need to create OAuth app on github.com
NextAuth.js installed	NEXT	Depends on GitHub OAuth keys
API routes	PENDING	After auth is working
Dashboard UI	PENDING	After API routes
Search page	PENDING	Needs TMDB API key
Public profile page	PENDING	After dashboard
Deployed to Vercel	PENDING	Final step

NOTE: The Prisma 7 config is different from older versions. URLs go in prisma.config.ts using env() from prisma/config — NOT in schema.prisma. This is already set up correctly.

2. System Architecture

WatchDiary is a single Next.js repository. The frontend (React pages) and backend (API routes) live in the same project. Prisma connects to PostgreSQL on Supabase. NextAuth handles sessions. TMDB provides movie/series/anime metadata and posters.

How the layers connect:



Folder Structure:

```
watchlist/
├── prisma/
│   ├── schema.prisma # Database models (already written)
│   ├── prisma.config.ts # Prisma 7 config with DB URLs (already set up)
│   └── .env # Secret keys (never commit to GitHub)
├── src/
│   ├── app/
│   │   ├── layout.tsx # Root layout (navbar, providers)
│   │   ├── page.tsx # Landing page
│   │   ├── login/page.tsx # Login page with OAuth buttons
│   │   ├── dashboard/page.tsx # Personal diary (protected)
│   │   ├── search/page.tsx # Search movies via TMDB
│   │   └── profile/[username]/page.tsx # Public shareable profile
│   ├── api/
│   │   ├── auth/[...nextauth]/route.ts # NextAuth handler
│   │   ├── entries/route.ts # GET + POST entries
│   │   ├── entries/[id]/route.ts # PUT + DELETE entry
│   │   ├── search/route.ts # TMDB proxy
│   │   └── profile/[username]/route.ts # Public profile data
│   ├── components/
│   │   ├── Navbar.tsx
│   │   ├── EntryCard.tsx
│   │   ├── AddEntryModal.tsx
│   │   └── StatsBar.tsx
│   └── lib/
```

```
■ ■■■ prisma.ts # Prisma client singleton
■ ■■■ auth.ts # NextAuth config
■ ■■■ tmdb.ts # TMDB API helper
```

3. Database Schema

Already set up and pushed to Supabase. Here is the complete schema for reference. You should NOT need to change this unless adding new features.

prisma/schema.prisma — complete file:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id String @id @default(cuid())
  name String?
  email String @unique
  image String?
  username String? @unique
  createdAt DateTime @default(now())
  entries Entry[]
  accounts Account[]
  sessions Session[]
}

model Entry {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  title String
  type EntryType
  status WatchStatus
  rating Int?
  notes String?
  poster String?
  tmdbId Int?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

enum EntryType { MOVIE SERIES ANIME }
enum WatchStatus { WATCHING COMPLETED DROPPED }

model Account {
  id String @id @default(cuid())
  userId String
  type String
  provider String
  providerAccountId String
  access_token String? @db.Text
  refresh_token String? @db.Text
}
```

```

user User @relation(fields: [userId], references: [id], onDelete: Cascade)
@@unique([provider, providerAccountId])
}

model Session {
  id String @id @default(cuid())
  sessionToken String @unique
  userId String
  expires DateTime
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

```

prisma.config.ts — complete file:

```

import "dotenv/config"
import path from "path"
import { defineConfig, env } from "prisma/config"

export default defineConfig({
  schema: path.join("prisma", "schema.prisma"),
  datasource: {
    url: env("DATABASE_URL"),
    directUrl: env("DIRECT_URL"),
  },
})

```

.env file — complete contents:

```

DATABASE_URL="postgresql://postgres.arxmvziepsgrsgdsshke:PASSWORD@aws-1-ap-south-1.pooler.supabase.com:6543/postgres?pgbouncer=true"
DIRECT_URL="postgresql://postgres.arxmvziepsgrsgdsshke:PASSWORD@aws-1-ap-south-1.pooler.supabase.com:5432/postgres"
GITHUB_ID="your-github-client-id"
GITHUB_SECRET="your-github-client-secret"
NEXTAUTH_URL="http://localhost:3000"
NEXTAUTH_SECRET="watchdiary_secret_2026"
TMDB_API_KEY="your-tmdb-api-key"

```

NOTE: Replace PASSWORD with cheems10072026 (or your actual password). Never share this file or push it to GitHub.

4. Step-by-Step Build Order

Follow this exact order. Each step depends on the one before it. Do not skip steps or build out of order.

#	Phase	What to build	How to test
1	Database + Prisma	Schema written, tables pushed to Supabase via npx prisma db push	Done! Prisma studio shows tables
2	GitHub OAuth	Create OAuth app on github.com/settings/developers	Sign in Client ID, Secret on login page
3	NextAuth setup	Install next-auth, write src/lib/auth.ts and src/api/auth/[...nextauth]/route.ts	Can sign in with GitHub, session works
4	Prisma client	Write src/lib/prisma.ts singleton so API routes can query the database	Importing prisma
5	Entries API	Write GET + POST in src/api/entries/route.ts	Postman/browser returns entry data
6	Edit/Delete API	Write PUT + DELETE in src/api/entries/[id]/route.ts	Can update and delete entries via API
7	TMDB API	Get free API key from themoviedb.org, write src/lib/tmdb.ts and src/api/search/route.ts	Search returns api/search/results
8	Dashboard UI	Build src/app/dashboard/page.tsx with entry grid, filters, can add, edit, delete entries	Can add, edit, delete entries in browser
9	Search page	Build src/app/search/page.tsx with TMDB search and search input	Search input shows anime results with posters
10	Public profile	Build src/api/profile/[username]/route.ts and src/app/profile/[username]/page.tsx	Visit profile page, not being logged in
11	Polish	Loading states, empty states, error messages, mobile responsiveness	App feels smooth and complete
12	Deploy	Push to GitHub, connect Vercel, add env vars in Vercel dashboard	Live dashboard from any device

5. Step 2 — GitHub OAuth Setup

This is the NEXT step to do right now. Follow exactly:

5.1 Create the GitHub OAuth App

1. Go to: <https://github.com/settings/developers>
2. Click OAuth Apps then New OAuth App
3. Fill in the form:

```
Application name: WatchDiary
Homepage URL: http://localhost:3000
Authorization callback URL: http://localhost:3000/api/auth/callback/github
```

4. Click Register application
5. Click Generate a new client secret
6. Copy both Client ID and Client Secret

5.2 Add keys to .env

Run these in PowerShell terminal:

```
Add-Content .env 'GITHUB_ID="paste-your-client-id-here"'
Add-Content .env 'GITHUB_SECRET="paste-your-client-secret-here"'
Add-Content .env 'NEXTAUTH_URL="http://localhost:3000"'
Add-Content .env 'NEXTAUTH_SECRET="watchdiary_secret_2026"'
```

5.3 Install NextAuth

```
npm install next-auth @next-auth/prisma-adapter
```

5.4 Create src/lib/prisma.ts

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = global as unknown as { prisma: PrismaClient }

export const prisma =
  globalForPrisma.prisma || new PrismaClient({ log: ['query'] })

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

5.5 Create src/lib/auth.ts

```
import NextAuth from 'next-auth'
import GitHubProvider from 'next-auth/providers/github'
import { PrismaAdapter } from '@next-auth/prisma-adapter'
import { prisma } from './prisma'

export const authOptions = {
  adapter: PrismaAdapter(prisma),
  providers: [
```

```

GitHubProvider({
  clientId: process.env.GITHUB_ID!,
  clientSecret: process.env.GITHUB_SECRET!,
}),
],
callbacks: {
  async session({ session, user }: any) {
    session.user.id = user.id
    return session
  },
  async signIn({ user }: any) {
    if (!user.email) return false
    const existing = await prisma.user.findUnique({
      where: { email: user.email }
    })
    if (!existing?.username) {
      const base = user.email.split('@')[0].replace(/^[^a-z0-9]/gi, '')
      await prisma.user.update({
        where: { email: user.email },
        data: { username: base }
      })
    }
    return true
  },
},
pages: { signIn: '/login' },
secret: process.env.NEXTAUTH_SECRET,
}
export default NextAuth(authOptions)

```

5.6 Create src/api/auth/[...nextauth]/route.ts

First create the folder: src/api/auth/[...nextauth]/

Then create route.ts inside it:

```

import NextAuth from 'next-auth'
import { authOptions } from '@lib/auth'
const handler = NextAuth(authOptions)
export { handler as GET, handler as POST }

```

6. Step 3 — API Routes

After auth is working, build these API routes one by one. Test each one before moving to the next.

6.1 GET + POST /api/entries

File: src/api/entries/route.ts

```
import { getServerSession } from 'next-auth'
import { authOptions } from '@lib/auth'
import { prisma } from '@lib/prisma'
import { NextResponse } from 'next/server'

export async function GET() {
  const session = await getServerSession(authOptions)
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  const entries = await prisma.entry.findMany({
    where: { userId: session.user.id },
    orderBy: { updatedAt: 'desc' },
  })
  return NextResponse.json(entries)
}

export async function POST(req: Request) {
  const session = await getServerSession(authOptions)
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  const body = await req.json()
  const { title, type, status, rating, notes, tmdbId, poster } = body
  if (!title || !type || !status) {
    return NextResponse.json({ error: 'Missing fields' }, { status: 400 })
  }
  const entry = await prisma.entry.create({
    data: { userId: session.user.id, title, type, status, rating, notes, tmdbId, poster }
  })
  return NextResponse.json(entry, { status: 201 })
}
```

6.2 PUT + DELETE /api/entries/[id]

File: src/api/entries/[id]/route.ts

```
export async function PUT(req: Request, { params }: { params: { id: string } }) {
  const session = await getServerSession(authOptions)
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  const existing = await prisma.entry.findUnique({ where: { id: params.id } })
  if (!existing || existing.userId !== session.user.id) {
    return NextResponse.json({ error: 'Not found' }, { status: 404 })
  }
  const body = await req.json()
  const updated = await prisma.entry.update({
    where: { id: params.id },
```

```

data: { status: body.status, rating: body.rating, notes: body.notes }
})
return NextResponse.json(updated)
}

export async function DELETE(_: Request, { params }: { params: { id: string } }) {
const session = await getServerSession(authOptions)
if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
const existing = await prisma.entry.findUnique({ where: { id: params.id } })
if (!existing || existing.userId !== session.user.id) {
return NextResponse.json({ error: 'Not found' }, { status: 404 })
}
await prisma.entry.delete({ where: { id: params.id } })
return NextResponse.json({ success: true })
}

```

6.3 TMDb Search Proxy

File: src/api/search/route.ts

```

export async function GET(req: Request) {
const { searchParams } = new URL(req.url)
const q = searchParams.get('q')
if (!q) return NextResponse.json({ results: [] })
const url = `https://api.themoviedb.org/3/search/multi?query=${encodeURIComponent(q)}&api_key=${process.env.TMDB_API_KEY}`
const res = await fetch(url)
const data = await res.json()
const results = data.results.slice(0, 8).map((item: any) => ({
tmdbId: item.id,
title: item.title || item.name,
type: item.media_type,
poster: item.poster_path ? `https://image.tmdb.org/t/p/w200${item.poster_path}` : null,
year: (item.release_date || item.first_air_date || '').slice(0, 4),
}))
return NextResponse.json({ results })
}

```

7. Step 4 — UI Pages

Build these pages in order. Each one is a React component in the `src/app/` directory.

7.1 Login Page — `src/app/login/page.tsx`

```
'use client'
import { signIn } from 'next-auth/react'
export default function LoginPage() {
  return (
    <div className="min-h-screen flex items-center justify-center">
      <div className="bg-white p-8 rounded-2xl shadow-sm border w-full max-w-sm">
        <h1 className="text-2xl font-semibold text-center mb-2">WatchDiary</h1>
        <p className="text-gray-500 text-center mb-8">Track every movie, series, and anime</p>
        <button
          onClick={() => signIn('github', { callbackUrl: '/dashboard' })}
          className="w-full bg-gray-900 text-white py-3 rounded-xl font-medium"
        >
          Sign in with GitHub
        </button>
      </div>
    </div>
  )
}
```

7.2 Dashboard Page — `src/app/dashboard/page.tsx`

```
'use client'
import { useState, useEffect } from 'react'
import { useSession } from 'next-auth/react'
import { redirect } from 'next/navigation'
export default function Dashboard() {
  const { data: session, status } = useSession()
  const [entries, setEntries] = useState([])
  const [filter, setFilter] = useState('ALL')
  const [showModal, setShowModal] = useState(false)
  if (status === 'unauthenticated') redirect('/login')
  useEffect(() => {
    fetch('/api/entries').then(r => r.json()).then(setEntries)
  }, [])
  const filtered = entries.filter((e: any) =>
    filter === 'ALL' || e.type === filter || e.status === filter
  )
  return (
    <div className="max-w-5xl mx-auto px-4 py-8">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-2xl font-semibold">My Diary</h1>
        <button onClick={() => setShowModal(true)}>

```

```

className="bg-blue-600 text-white px-4 py-2 rounded-xl">
+ Add title
</button>
</div>
<StatsBar entries={entries} />
<FilterBar filter={filter} setFilter={setFilter} />
<div className="grid grid-cols-2 md:grid-cols-4 gap-4 mt-6">
{filtered.map((e: any) => <EntryCard key={e.id} entry={e} />)}
</div>
</div>
)
}

```

7.3 Public Profile — src/app/profile/[username]/page.tsx

```

export default async function ProfilePage({ params }: { params: { username: string } }) {
const res = await fetch(`${process.env.NEXTAUTH_URL}/api/profile/${params.username}`)
if (!res.ok) return <div>Profile not found</div>
const user = await res.json()

return (
<div className="max-w-4xl mx-auto px-4 py-8">
<div className="flex items-center gap-4 mb-8">
<img src={user.image} className="w-16 h-16 rounded-full" />
<div>
<h1 className="text-2xl font-semibold">{user.name}</h1>
<p className="text-gray-500">@{user.username}</p>
</div>
</div>
<div className="grid grid-cols-2 md:grid-cols-4 gap-4">
{user.entries.map((e: any) => <EntryCard key={e.id} entry={e} />)}
</div>
</div>
)
}

```

8. Step 5 — Deploy to Vercel

Do this last, after everything works locally.

8.1 Push to GitHub

```
git add .  
git commit -m 'initial commit'  
git remote add origin https://github.com/yourusername/watchdiary.git  
git push -u origin main
```

NOTE: Make sure `.env` is in your `.gitignore` file so your secrets never get pushed to GitHub.

8.2 Connect Vercel

1. Go to vercel.com and sign in with GitHub
2. Click New Project and select your watchdiary repository
3. Go to Settings then Environment Variables
4. Add every variable from your `.env` file one by one
5. Click Deploy — Vercel builds and hosts the app automatically

8.3 Update GitHub OAuth for production

After deploying, go back to github.com/settings/developers and update your OAuth app:

Homepage URL: `https://your-app.vercel.app`

Authorization callback URL: `https://your-app.vercel.app/api/auth/callback/github`

9. Known Issues and Fixes

Error	Cause	Fix
url is no longer supported in schema.prisma	Prisma 4 breaking change	Remove url from schema.prisma, put it in prisma.config.ts using env
Cannot resolve environment variable DATABASE_URL	Quotes stripped by PowerShell	Use Set-Content instead of echo to write .env
Tenant or user not found	Wrong region in pooler URL	Use aws-1-ap-south-1 not aws-0-ap-southeast-1
P1001: Can't reach database server	Wrong URL or Supabase paused	Check .env has correct URL, check Supabase project is active
P1013: scheme not recognized	Special characters in password	Use simple password with letters and numbers only
ERR_STREAM_PREMATURE_CLOSE	Close Prisma Studio	Safe to ignore, Studio still works

Important: How to write .env on Windows

Always use Set-Content in PowerShell, never echo:

```
# CORRECT way to write .env on Windows PowerShell:
Set-Content .env 'DATABASE_URL="postgresql://..."'
Add-Content .env 'GITHUB_ID="your-id"'

# WRONG - echo strips the quotes:
echo DATABASE_URL="postgresql://..." > .env
```

10. Quick Reference

Commands you will use repeatedly:

```
npm run dev # Start local dev server at localhost:3000
npx prisma db push # Push schema changes to Supabase
npx prisma studio # Open visual database browser at localhost:5555
npx prisma generate # Regenerate Prisma client after schema changes
npm run build # Test production build before deploying
cat .env # Check what is in your .env file
```

Your project details:

```
Project folder: C:\Users\Almeda\Desktop\watchlist
Supabase project: arxmvmziepsgrsgdsshke
Supabase region: aws-1-ap-south-1
Local URL: http://localhost:3000
Prisma Studio: http://localhost:5555
DB password: cheems10072026 (change this later)
```

What to tell interviewers:

1. I built every layer myself — database schema, auth, REST API, and React UI
2. I used Prisma as the ORM because it gives type-safe queries and easy migrations
3. The TMDB search goes through my own API route so the key is never exposed to the browser
4. I handle ownership checks before every update and delete — users can only modify their own entries
5. The public profile is server-rendered for fast load, the dashboard is client-rendered for interactivity

End of Document — WatchDiary Build Blueprint v1.0 — 2026